

Pypomp: Inference for partially observed Markov process models in Python with JAX DRAFT IN PROGRESS

Aaron J. Abkemeier¹

Jun Chen²

Kevin Tan³

Jesse Wheeler⁴

Bo Yang⁵

Kunyang He¹

Jonathan Terhorst¹

Aaron A. King

Edward L. Ionides¹

AARONABK@UMICH.EDU

CHENJUNC@UMICH.EDU

KEVTAN@WHARTON.UPENN.EDU

JESSEWHEELER@ISU.EDU

YBB@UMICH.EDU

KUNYANGHE@UMICH.EDU

JONTH@UMICH.EDU

IONIDES@UMICH.EDU

¹University of Michigan, Department of Statistics; ²Duke University, Department of Statistics;
³University of Pennsylvania, Department of Statistics; ⁴Idaho State University, Department of
Mathematics; ⁵Columbia University, Department of Marketing

Editor:

Abstract

Model development and parameter estimation for non-Gaussian partially observed Markov process (POMP) mechanistic models are fundamental challenges in a variety of fields including epidemiology, ecology, and finance. Particle filter methods can provide statistically efficient inference for highly nonlinear POMP models, but their practical application is limited by high computational demands. We introduce Pypomp, a high-performance Python library for statistical inference using POMP models. Pypomp includes state-of-the-art particle filtering algorithms to take advantage of GPU hardware, just-in-time computation, and automatic differentiation accessed using JAX. Basic inference algorithms run up to 1000 times faster than on a CPU core, and advanced algorithms provide further acceleration [EL: THIS SEEMS LIKE A BIG CLAIM, BUT IF WE CAN MAKE AND ADEQUATELY SUPPORT A BIG CLAIM, WE SHOULD DO SO]. Pypomp provides a comprehensive interface that streamlines model construction, fitting, and analysis, allowing practitioners to develop and evaluate complex models with minimal implementation overhead.

Keywords: pomp, JAX, GPU acceleration, likelihood-based inference

1 Introduction

Partially Observed Markov Process (POMP) models, also known as state-space models or hidden Markov models, provide a flexible mechanistic framework for modeling dynamic systems where the underlying state process is not directly observed. The particle filter (Arulampalam et al., 2002), also known as sequential Monte Carlo, enables evaluation of the likelihood for POMP models. The extension of particle filtering to iterated filtering (Ionides et al., 2006, 2015) permits maximum likelihood estimation. A defining strength of

these methods is that they are performed by simulating from the process model instead of evaluating exact transition probabilities (a property known as plug-and-play (Bretó et al., 2009)), which makes them suitable for examining a wide range of models unconstrained by a need for mathematical convenience. Remarkably, algorithms based on particle filtering permit statistically efficient, likelihood-based inference in various practical situations even when the likelihood can be approximated only by Monte Carlo methods. Consequently, particle-filter-based methods for POMP models find application in epidemiology (Mietchen et al., 2024; Fox et al., 2022; Wen et al., 2024), ecology (Auger-Méthé et al., 2021; Marino et al., 2019; Blackwood et al., 2013), finance (Bretó, 2014), and other domains where mechanistic dynamic modeling is desired.

Pypomp is a Python library combining advances in hardware, software, and algorithms to provide novel capabilities for POMP models, with an emphasis on inference via particle filters. Pypomp is built using JAX (Bradbury et al., 2018), which provides access to massively parallel hardware such as GPUs. Pypomp inherits software design principles from the successful R `pomp` package (King et al., 2016), providing interoperable frameworks for the development of new models and new statistical methods. Pypomp implements algorithmic advances that are required to take advantage of the GPU and automatic differentiation (AD) capabilities of JAX. AD for particle filters is a current research topic, and Pypomp implements a state-of-the-art approach that can simultaneously control the bias and variance (Tan et al., 2024). The particle filter involves many simulated random variables, and traditional rejection sampling algorithms for commonly used distributions are not well suited to GPU architectures. Pypomp addresses this by incorporating alternative sampling algorithms.

For systems taking values in a small discrete state space, or for systems well modeled by a linear Gaussian process, or sufficiently close to linear Gaussian that approximations can take advantage of the closed-form Gaussian filtering equations, it is not necessary to use the particle filter. In these cases, the `Dynamax` library (Linderman et al., 2025) provides a JAX-based solution. As dynamic models increase in complexity and nonlinearity, the particle filter methods in Pypomp become increasingly valuable. This makes Pypomp especially relevant to applications in ecology and epidemiology. The plug-and-play property of the particle filter makes Pypomp a convenient exploratory tool, enabling rapid model development even in situations where alternative methods are applicable. Pypomp incorporates advances in high-dimensional particle filter inference for panel data (Bretó et al., 2020) to address a theoretical limitation of the particle filter, that it scales poorly with state space dimension.

[EL: WITH SUITABLE DISCLAIMERS, WE HAVE OBTAINED ERC VALUES OF AROUND 1000, RIGHT? THAT'S ANOTHER ORDER OF MAGNITUDE BIGGER. IT NEEDS EXTRA VMAPPING, SO THE STATEMENT WOULD HAVE TO BE CAREFUL, BUT PRESUMABLY ONE COULD DRAFT SOMETHING CORRECT THAT DISPLAYS THE BEST RATIO WE HAVE.]

On a single GPU, Pypomp runs IF2 12 to 61 times faster than the R packages `pomp` and `panelPomp` running on a 36-core CPU node, which is the throughput of 420 to 2,208 CPU cores running R (Section 4). This enables plug-and-play likelihood-based inference on datasets that would otherwise be intractably large. In Section 4.1, we fit a panel POMP model to the England and Wales measles dataset of Korevaar et al. (2020), which contains

over one million observations from 1,422 towns and cities. One GPU completed the fit in 29 hours, whereas we estimate that the same computation would take about 4.5 months with the R package `panelPomp` (Bretó et al., 2020).

[EL: A BRAINSTORMING IDEA: IN THE SMC READING GROUP, I STARTED USING THE PHRASE “MICHIGAN NOTATION” TO EXPLAIN OUR COHERENT SET OF NAMES AND NOTATIONS THAT IS SOMEWHAT DIFFERENT TO THE DIVERSE TERMINOLOGIES FOUND ELSEWHERE. I’M NOT SURE IF THIS IS THE RIGHT PLACE, BUT MAYBE IT IS. ANYWAY, HERE’S A GO]

2 POMP Modeling: The Michigan notation

Diverse terminologies and notations have evolved for POMP models in different communities. Pypomp uses the Michigan notation that arose from the program to build a unifying software environment to represent a wide range of proposed statistical methodologies in the R `pomp` package. Distinguishing features are: (i) t is used to represent continuous time, allowing proper handling scientific units and situations where the latent variable is defined not just at times it is observed; (ii) $\{t_n, n = 1, \dots, N\}$ is a discrete sequence of N measurement times; (iii) ranges are upper-case Roman letters with the corresponding lower-case often used as an index; (iv) joint and conditional densities are defined using appropriate subscripts and avoiding overloading, for example, $f_{Y_n|X_n}(y_n|x_n;\theta)$ denotes the conditional density of Y_n given X_n ; (v) sequences are denoted by colons, for example, $\{t_n, n = 1, \dots, N\}$ is written as $t_{1:N}$. [EL: TO BE CONTINUED, IF WE GO THIS ROUTE].

A POMP model includes a random vector of observed data $Y_{1:N}$, and the observed values are denoted as $y_{1:N}^*$. The data are generated based on the values of the unobserved process, $\{X(t), t_0 \leq t \leq t_N\}$, where $t_{1:N}$ are the times at which $Y_{1:N}$ are observed, t_0 is the time at which the initial state is defined, and $X_n := X(t_n)$. We suppose the model has a joint distribution $f_{Y_{1:N}, X_{0:N}}(y_{1:N}, x_{0:N}; \theta)$, where θ is a set of parameters, and Y_n is independent of $Y_m, X_m, m \neq n$, conditional on X_n .

Although evaluating the joint distribution is potentially intractable, we can still implement plug-and-play POMP methodology. For example, particle filtering (Arulampalam et al., 2002) can be used to estimate the marginal likelihood $f_{Y_{1:N}}(y_{1:N}; \theta)$, and Iterated Filtering 2 (Ionides et al., 2015) can be used to estimate the maximum likelihood estimate (MLE) of θ , both without an explicit joint distribution or the explicit transition density $f_{X_{n+1}|X_n}(x_{n+1}|x_n; \theta)$. For these methods, it is sufficient to specify a collection of functions associated with the model’s conditional distribution densities:

- $X_0 \sim f_{X_0}(x_0; \theta)$: The simulator for the initial state of the latent process.
- $X_{n+1} \sim f_{X_{n+1}|X_n}(x_{n+1}|x_n; \theta)$: The simulator for the transition of the latent process.
- $f_{Y_n|X_n}(y_n|x_n; \theta)$: The measurement density function.

To implement POMP models and methods in `pypomp`, the user can provide data and programmatic descriptions of the above functions as arguments to a `Pomp` object. With these necessary components bundled together, the user can perform inference on the model using `pypomp`’s methods.

3 POMP Inference Methods in pypomp

`pypomp` implements several likelihood-based inference methods, all of which leverage the plug-and-play property (Ionides et al., 2006) and JAX-enabled hardware acceleration.

1. **Particle Filter (`pfilter`)**: The standard sequential Monte Carlo (SMC) algorithm for likelihood evaluation and state estimation.
2. **Iterated Filtering (`mif`)**: An implementation of the IF2 algorithm (Ionides et al., 2015) for maximum likelihood estimation. It combines particle filtering with sequential random walk perturbations and a cooling schedule to explore the parameter space.
3. **Automatic Differentiation (`mop` and `train`)**: `pypomp` includes a differentiable particle filter (MOP- α , Tan et al. (2024)) that enables the use of JAX’s automatic differentiation. The `train()` method uses these gradients for local optimization.

These algorithms can be run as methods of the `Pomp` class. Users can define their models by providing JAX-compatible functions for each component of the POMP model. For example, the S&P500 model included in the package defines the state transition simulator as follows:

```
import jax
import jax.numpy as jnp
from pypomp.types import StateDict, ParamDict, CovarDict, TimeFloat, StepSizeFloat, RNGKey

def rproc(
    X_: StateDict, theta_: ParamDict, key: RNGKey, covars: CovarDict,
    t: TimeFloat, dt: StepSizeFloat
):
    V, S = X_["V"], X_["S"]
    mu, kappa, theta = theta_["mu"], theta_["kappa"], theta_["theta"]
    xi, rho, y_prev = theta_["xi"], theta_["rho"], covars["y_prev"]
    dZ = jax.random.normal(key)
    dWs = (y_prev - mu + 0.5 * V) / jnp.sqrt(V)
    dWv = rho * dWs + jnp.sqrt(1 - rho**2) * dZ
    S = S + S * (mu + jnp.sqrt(jnp.maximum(V, 1e-32))) * dWs
    V = V + kappa * (theta - V) + xi * jnp.sqrt(V) * dWv
    V = jnp.maximum(V, 1e-32)
    return {"V": V, "S": S}
```

Internally, `pypomp` takes a user-defined function such as this, vectorizes it over the particles and replications of a method (it is prudent to run these methods from multiple values of θ), and JIT compiles it using JAX, achieving high performance by parallelizing computation across GPU cores.

The following example demonstrates a possible inference workflow, expressed in just a few lines: starting with global exploration via IF2, followed by gradient-based local refinement, and concluding with a final likelihood evaluation.

```
import pypomp as pp
spx_obj = pp.models.spx()
key = jax.random.key(1)
rw_sd = pp.RWSigma( # Random walk standard deviations for IF2
    sigmas={"mu": 0.02, "kappa": 0.02, "theta": 0.02,
            "xi": 0.02, "rho": 0.02, "V_0": 0.1},
    init_names=["V_0"],
).geometric_cooling(a=0.5)
spx_obj.mif(rw_sd=rw_sd, J=1000, M=10, key=key) # IF2
eta = pp.LearningRate(
    {name: 0.001 for name in spx_obj.canonical_param_names}
).geometric_decay(decay_rate=0.5, M=10) # Learning rates
spx_obj.train(J=1000, M=10, eta=eta, optimizer=pp.Adam()) # Gradient descent
spx_obj.pfilter(J=1000, reps=10) # Particle filter
```

Each method call generates an object which contains details including log-likelihood estimates, parameter traces, execution time, and the arguments passed to the method. These objects are then appended to `spx_obj.results_history` and can be interfaced with through helper functions, such as the `traces` method, which returns a long-format data frame containing parameter and log-likelihood traces across all method calls.

Methods like `mif` and `train` achieve high performance by internally running JIT-compiled JAX functions from the submodule `pypomp.functional`. The user may import these functions directly to build custom inference procedures.

The open source repository is available at <https://github.com/pypomp/pypomp>, and full documentation is available at <https://pypomp.readthedocs.io>.

4 Performance

We benchmarked Pypomp against the R packages `pomp` and `panelPomp` using the quantitative test suite at <https://github.com/pypomp/quant> (commit d8559d6). Table 1 reports wall-clock times for the same workload on three platforms: R on a 36-core CPU node, Pypomp on a 36-core CPU node, and Pypomp on one RTX PRO 6000 Blackwell Server Edition GPU. The runs used Pypomp 1.0.0rc1, `pomp` 6.3, and `panelPomp` 1.7.0.0. On the GPU, Pypomp runs IF2 between 12 and 61 times faster than R. R divided each workload evenly across its 36 cores, so a single core would take 36 times as long, and one GPU matches the IF2 throughput of 420 to 2,208 CPU cores running R. Pypomp on the CPU is faster than R for the cholera and S&P 500 models but slower for the measles model, whose Poisson, binomial, and gamma draws are expensive for JAX on a CPU. The GPU times depend on the approximate inverse-CDF samplers in `pypomp.random`: with the standard `jax.random` samplers, IF2 for the London measles model took 29 times longer on the same GPU.

Table 1: Wall-clock time in seconds for IF2 and for particle filter (PF) evaluation of the IF2 estimates. Each IF2 row runs 36 searches from the same starting points on every platform. Pypomp PF times include JIT compilation. Speedup is the R time divided by the Pypomp GPU time. Throughput relative to one core is the speedup multiplied by the 36 R cores: the number of CPU cores running R needed to match one GPU. The 4-town panel PF comparisons are omitted because the two quant scripts filter different numbers of parameter sets, and no Pypomp CPU run was recorded for that model.

Model	Task	R (36 cores)	Pypomp (36 cores)	Pypomp (1 GPU)	Speedup	Throughput vs. 1 core
Cholera, Dhaka	IF2	1,566	398	30.4	51×	1,852×
	PF	359	68.7	9.1	39×	1,421×
Measles, London	IF2	2,923	4,484	47.6	61×	2,208×
	PF	895	1,544	19.5	46×	1,653×
Measles, 4-town panel	IF2	8,187	—	168	49×	1,759×
	PF	67.4	—	75.4	—	—
S&P 500	IF2	703	283	60.2	12×	420×
	PF	36.4	7.3	8.4	4×	156×

The S&P 500 model shows the smallest GPU advantage in Table 1 because its benchmark gives the GPU little work to do in parallel. It runs 36 searches of 1,000 particles and resamples after each of its 3,523 time steps, each of which involves little computation, so fixed per-step costs dominate the GPU time. Table 2 shows that the GPU’s throughput grows with the number of particles filtered at once. With 2,000 searches of 1,000 particles, one GPU matches the IF2 throughput of 5,147 CPU cores running R. Adding searches raises throughput more than adding particles to each search, and with 120 searches, throughput falls once each search has more than 15,000 particles. Peak GPU memory grows more slowly than the number of particles and stays below 1 GB in every configuration.

Table 2: IF2 time per iteration and peak GPU memory for the S&P 500 model on one GPU as the number of searches (chains) and particles per search grow. The first row is the benchmark in Table 1, whose memory use was not recorded. R cores matched is the number of CPU cores running R needed to match the GPU’s IF2 throughput, based on the R time in Table 1. Each configuration ran 50 IF2 iterations, and time per iteration includes JIT compilation.

Chains	Particles per chain	Total particles	IF2 s/iteration	Peak memory (MB)	R cores matched
36	1,000	36,000	0.30	—	420

Table 2: IF2 time per iteration and peak GPU memory for the S&P 500 model on one GPU as the number of searches (chains) and particles per search grow. The first row is the benchmark in Table 1, whose memory use was not recorded. R cores matched is the number of CPU cores running R needed to match the GPU’s IF2 throughput, based on the R time in Table 1. Each configuration ran 50 IF2 iterations, and time per iteration includes JIT compilation.

Chains	Particles per chain	Total particles	IF2 s/iteration	Peak memory (MB)	R cores matched
250	1,000	250,000	0.47	82	1,884
500	1,000	500,000	0.64	128	2,731
750	1,000	750,000	0.71	195	3,688
1,000	1,000	1,000,000	0.83	226	4,243
1,500	1,000	1,500,000	1.06	391	4,959
2,000	1,000	2,000,000	1.36	451	5,147
120	5,000	600,000	0.77	128	2,744
120	10,000	1,200,000	1.08	256	3,900
120	15,000	1,800,000	1.47	354	4,292
120	20,000	2,400,000	2.08	448	4,063
120	30,000	3,600,000	3.26	693	3,882
120	40,000	4,800,000	5.04	842	3,346

4.1 Fitting all towns in England and Wales

To show the scale of problem that this acceleration makes feasible, we fit the measles model of He et al. (2010) to weekly case reports from all 1,422 towns and cities in England and Wales between 1950 and 1963 (Korevaar et al., 2020), treating every parameter as specific to each town. We ran 12 panel IF2 searches, each with 10,000 particles for 200 iterations, and evaluated each endpoint with 36 particle filter replicates of 5,000 particles. We then combined the best estimate for each town across the searches (Bretó et al., 2020) and evaluated the combined estimate with another 36 replicates. Table 3 gives the time for each stage on one RTX PRO 6000 Blackwell Server Edition GPU with Pypomp 1.0.0.

For comparison, we estimated how long the same computation would take with `panelPomp` on a 36-core node. We measured `panelPomp`’s cost per particle, per IF2 iteration, and per town on the 4-town panel in Table 1, which covers the same period with the same daily time step, and scaled it linearly to the full computation. IF2 searches run sequentially, so R can use at most one core per search, and the extra cores only help with the particle filter evaluations. By this estimate, R would need about 4.5 months, 114 times longer than the 29 hours taken on the GPU. On a single core, R would need about 4.8 years, so the GPU delivered the throughput of about 1,439 CPU cores running R.

Table 3: Wall-clock time for fitting the measles model to all towns in England and Wales. Pypomp times were measured on one GPU. R times are estimates for `panelPomp` on a 36-core CPU node, extrapolated from Table 1.

Stage	Pypomp (1 GPU)	R <code>panelPomp</code> , 36 cores (estimated)
IF2	27.3 hours	134.7 days
PF, all search endpoints	99 minutes	3.3 days
PF, combined estimate	10 minutes	6.7 hours
Total	29.1 hours	138.3 days

5 Discussion

References

- M.S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, February 2002. ISSN 1053587X. doi: 10.1109/78.978374.
- Marie Auger-Méthé, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, Joanna Mills Flemming, Anders Nielsen, Giovanni Petris, and Len Thomas. A guide to state-space modeling of ecological time series. *Ecological Monographs*, 91(4):e01470, 2021. doi: 10.1002/ecm.1470. URL <https://doi.org/10.1002/ecm.1470>.
- J. C. Blackwood, D. G. Streicker, S. Altizer, and P. Rohani. Resolving the roles of immunity, pathogenesis, and immigration for rabies persistence in vampire bats. *Proceedings of the National Academy of Sciences of the United States of America*, 110(51):20837–20842, December 2013. doi: 10.1073/pnas.1308817110. URL <https://doi.org/10.1073/pnas.1308817110>.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018.
- Carles Bretó. On idiosyncratic stochasticity of financial leverage effects. *Statistics & Probability Letters*, 91:20–26, 2014. doi: 10.1016/j.spl.2014.04.003. URL <https://doi.org/10.1016/j.spl.2014.04.003>.
- Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, March 2009. ISSN 1932-6157, 1941-7330. doi: 10.1214/08-AOAS201.
- Carles Bretó, Edward L. Ionides, and Aaron A. King. Panel Data Analysis via Mechanistic Models. *Journal of the American Statistical Association*, 115(531):1178–1188, June 2020. ISSN 0162-1459, 1537-274X. doi: 10.1080/01621459.2019.1604367.
- S. J. Fox, M. Lachmann, M. Tec, R. Pasco, S. Woody, Z. Du, X. Wang, T. A. Ingle, E. Javan, M. Dahan, K. Gaither, M. E. Escott, S. I. Adler, S. C. Johnston, J. G. Scott,

- and L. A. Meyers. Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119, 2022. doi: 10.1073/pnas.2111870119. URL <https://doi.org/10.1073/pnas.2111870119>.
- Daihai He, Edward L. Ionides, and Aaron A. King. Plug-and-play inference for disease dynamics: Measles in large and small populations as a case study. *Journal of The Royal Society Interface*, 7(43):271–283, February 2010. ISSN 1742-5689, 1742-5662. doi: 10.1098/rsif.2009.0151.
- E. L. Ionides, C. Breto, and A. A. King. Inference for nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 103(49):18438–18443, December 2006. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.0603181103.
- Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, January 2015. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.1410597112.
- Aaron A. King, Dao Nguyen, and Edward L. Ionides. Statistical Inference for Partially Observed Markov Processes via the *R* Package **pomp**. *Journal of Statistical Software*, 69(12), 2016. ISSN 1548-7660. doi: 10.18637/jss.v069.i12.
- Hannah Korevaar, C. Jessica Metcalf, and Bryan T. Grenfell. Structure, space and size: Competing drivers of variation in urban and rural measles transmission. *Journal of The Royal Society Interface*, 17(168):20200010, July 2020. doi: 10.1098/rsif.2020.0010.
- Scott W Linderman, Peter Chang, Giles Harper-Donnelly, Aleyna Kara, Xinglong Li, Gerardo Duran-Martin, and Kevin Murphy. Dynamax: A python package for probabilistic state space modeling with JAX. *Journal of Open Source Software*, 10(108):7069, 2025. doi: 10.21105/joss.07069.
- J. A. Jr. Marino, S. D. Peacor, D. B. Bunnell, H. A. Vanderploeg, S. A. Pothoven, A. K. Elgin, J. R. Bence, J. Jiao, and E. L. Ionides. Evaluating consumptive and nonconsumptive predator effects on prey density using field time-series data. *Ecology*, 100(3):e02583, March 2019. doi: 10.1002/ecy.2583. URL <https://doi.org/10.1002/ecy.2583>.
- Matthew S. Mietchen, Erin Clancey, Corrin McMichael, and Eric T. Lofgren. Estimating sars-cov-2 transmission parameters between coinciding outbreaks in a university population and the surrounding community, Jan 2024. URL <https://doi.org/10.1101/2024.01.10.24301116>.
- Kevin Tan, Giles Hooker, and Edward L. Ionides. Accelerated Inference for Partially Observed Markov Processes using Automatic Differentiation, July 2024.
- L. Wen, Y. Yin, Q. Li, Z. Peng, and D. He. Modeling the co-circulation of influenza and covid-19 in hong kong, china. *Advances in Continuous and Discrete Models*, 2024 (1):1–9, 2024. doi: 10.1186/s13662-024-03830-7. URL <https://doi.org/10.1186/s13662-024-03830-7>. Article 32.

6 Appendix

[AA: Include important numerical results here? Or refer to the quant repo?]